Factorial Time complexity using Recursion Tree Method

$$T(n) = \begin{cases} 1 & ,for\ (n = 0) \\ 1 & ,for\ (n = 1) \\ T(n-1) + T(1) & ,for\ (n > 1) \end{cases}$$

Or for

$$T(n) = \begin{cases} 1 & ,for\ (n = 0) \\ 1 & ,for\ (n = 1) \\ T(n-1) + 1 & ,for\ (n > 0) \end{cases}$$

Lets recall the program:

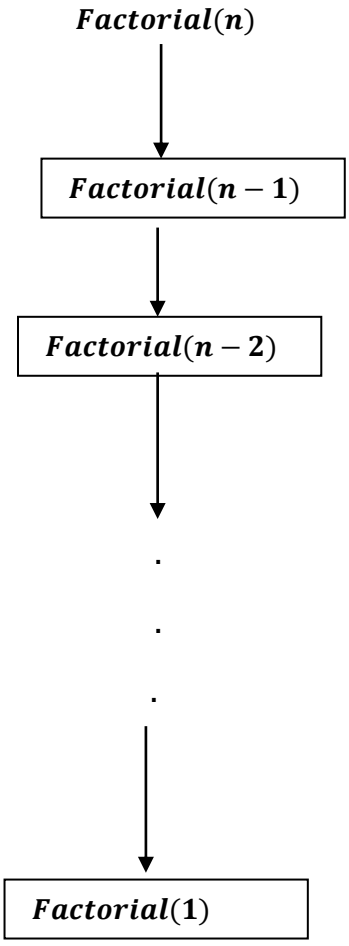
```
int factorial(int n)
{
    // Base case: if n is 0 or 1, return 1
    // This is because the factorial of 0 and 1 is 1
    if (n == 0 || n == 1)
    {
        return 1;
    }
```

```
    else
    {
        // Recursive case: n times factorial of n-1
        // This is the definition of factorial: the product
of an integer and all the integers below it
        return n * factorial(n - 1);
    }
}
```

The Work per Problem: ***While it waits for*** $factorial(n-1)$ ***to finish, what physical work does the CPU have to do? It just has to multiply the result by*** n***. Multiplying two numbers takes a tiny, fixed amount of CPU time. Because there are no loops, this work is constant time:*** $O(1)$***, which we call `C`.***

Therefore, the Work per Problem is Constant (C).

And recursion goes like :



$\boldsymbol{Level}$	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem\ (Print\ and\ Add)}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\mathbf{0}$	$\mathbf{1}$	$\boldsymbol{n}$	$\boldsymbol{C}$	$\mathbf{1} \times \boldsymbol{C} = \boldsymbol{C}$
$\mathbf{1}$	$\mathbf{1}$	$\boldsymbol{n-1}$	$\boldsymbol{C}$	$\mathbf{1} \times \boldsymbol{C} = \boldsymbol{C}$
$\mathbf{2}$	$\mathbf{1}$	$\boldsymbol{n-2}$	$\boldsymbol{C}$	$\mathbf{1} \times \boldsymbol{C} = \boldsymbol{C}$
. .	. .	. .	. .	. .
$\boldsymbol{n-1}$	$\mathbf{1}$	$\mathbf{1}(\boldsymbol{Base\ Case})$	$\boldsymbol{T}(\mathbf{1})$	$\mathbf{1} \times \boldsymbol{T}(\mathbf{1}) = \boldsymbol{T}(\mathbf{1})$

$\boldsymbol{Total\ Cost} = \boldsymbol{C} + \boldsymbol{C} + \boldsymbol{C} + \cdots (\boldsymbol{N-1})\boldsymbol{times} + \boldsymbol{T}(\mathbf{1}).$

$= \boldsymbol{C}(\boldsymbol{N-1}) + \boldsymbol{T}(\mathbf{1})$

$\boldsymbol{We\ know}, \boldsymbol{T}(\mathbf{1})\boldsymbol{is}\ \mathbf{1}\ \boldsymbol{instead\ lets\ write}\ \boldsymbol{T}(\mathbf{1}) = \boldsymbol{C_1}$,

$\boldsymbol{representing\ constant}.$

$= \boldsymbol{C}(\boldsymbol{N-1}) + \boldsymbol{C_1}$

$\boldsymbol{As}, \boldsymbol{C}\ \boldsymbol{and}\ \boldsymbol{C_1}\ \boldsymbol{both\ are\ constant\ Applying\ Big\ O}, \boldsymbol{we\ get}:$

$\boldsymbol{O}(\boldsymbol{N-1})$

$= \boldsymbol{O}(\boldsymbol{N})$